EXPERIMENT NO: 05

Aim: Simulate the functioning of Lamport's Logical Clock

Theory for Lamport's Logical Clock

Introduction

In a distributed system, there is no global clock, meaning that different processes operate independently without a synchronized reference time. This leads to challenges in determining the correct order of events, especially when processes communicate with each other asynchronously. Lamport's Logical Clock, proposed by Leslie Lamport in 1978, provides a mechanism to order events in a distributed system using a simple logical timestamping technique.

Concept of Lamport's Logical Clock

A Lamport Logical Clock is an integer value assigned to each event in a process, helping to establish a causal relationship between events. The algorithm ensures that if one event occurs before another, its timestamp is smaller.

Each process maintains a logical clock (LC) that follows these rules:

1. Internal Events: When a process performs an internal action, it increments its own clock.
2. Sending a Message: Before sending a message, a process increments its clock and attaches this timestamp to the message.
3. Receiving a Message: When a process receives a message, it updates its clock using the formula:

$$LC = \max(LC, \text{Received Timestamp}) + 1$$

1. This ensures that the receiver's logical clock is always ahead of any event that could have influenced it.

Rules of Lamport's Logical Clock

1. LC1: Monotonicity Rule
   - A process increments its clock for every internal event.
2. LC2: Send Rule
   - Before sending a message, a process increments its clock and sends the updated timestamp.
3. LC3: Receive Rule
   - When receiving a message, the process updates its clock to max(received timestamp, current clock) + 1.

These rules ensure a partial ordering of events in a distributed system.

Implementation Explanation

The experiment simulates three processes (p1, p2, p3) that:

- Perform internal events (increment their logical clock).
- Send messages to other processes (increment before sending).
- Receive messages and update their clock using the maximum rule.

This ensures that the logical timestamps maintain a causal ordering of events.

Use Cases

- Distributed Systems Synchronization Helps in orderingeventswithout requiring a global clock.
- Concurrency Control Used in distributeddatabasesto avoidconflicts.
- Distributed Debugging Helps in trackingandverifying thesequence of operations.

```
# Class to implement Lamport's Logical Clock
class LamportClock:
    def __init__(self, pid):
        """Initialize process with a unique process ID and clock set to 0."""
        self.pid = pid  # Process ID
        self.clock = 0  # Initialize logical clock to 0

    def event(self):
        """Simulates an internal event where the process performs an action."""
        self.clock += 1  # Increment the logical clock
        print(f"Process {self.pid}: Internal Event -> Clock = {self.clock}")

    def send_message(self, receiver):
        """Simulates sending a message to another process."""
        self.clock += 1  # Increment clock before sending message
        print(f"Process {self.pid}: Sent message to Process {receiver.pid} -> Clock = {self.clock}")
        receiver.receive_message(self.clock)  # Call receive_message on receiver with current clock

    def receive_message(self, received_timestamp):
        """Simulates receiving a message and updating the logical clock."""
        # Update clock to max(current clock, received timestamp) + 1
        self.clock = max(self.clock, received_timestamp) + 1
        print(f"Process {self.pid}: Received message -> Updated Clock = {self.clock}")

# Create three processes with unique process IDs
p1 = LamportClock(1)
p2 = LamportClock(2)
p3 = LamportClock(3)

# Simulate events and message passing between processes

p1.event()                # P1 performs an internal event (clock incremented by 1)
p1.send_message(p2)       # P1 sends a message to P2 (clock incremented before sending)

p2.event()                # P2 performs an internal event (clock incremented by 1)
p2.send_message(p3)       # P2 sends a message to P3 (clock incremented before sending)

p3.event()                # P3 performs an internal event (clock incremented by 1)
p3.send_message(p1)       # P3 sends a message to P1 (clock incremented before sending)
```

```
Process 1: Internal Event -> Clock = 1
Process 1: Sent message to Process 2 -> Clock = 2
Process 2: Received message -> Updated Clock = 3
Process 2: Internal Event -> Clock = 4
Process 2: Sent message to Process 3 -> Clock = 5
Process 3: Received message -> Updated Clock = 6
Process 3: Internal Event -> Clock = 7
Process 3: Sent message to Process 1 -> Clock = 8
Process 1: Received message -> Updated Clock = 9
```

Conclusion: Lamport’s Logical Clock provides a simple yet effective method to track event ordering in a distributed system. While it does not capture causality completely (since different events may have the same timestamp), it is widely used for synchronization, logging, and debugging in distributed environments.

```
class LamportClock:
    def __init__(self, pid):
        # Initialize process with a unique process ID and clock 0
        self.pid = pid
        self.clock = 0

    def event(self):
        """Simulates an internal event where the process performs an action."""
        self.clock += 1
        print(f"Process {self.pid}: Internal Event | Clock: {self.clock}")

    def send_message(self, receiver):
        """Simulates sending a message to another process."""
        self.clock += 1  # Increment clock before sending message
        print(f"Process {self.pid}: Sent message to Process {receiver.pid} | Clock: {self.clock}")
        receiver.receive_message(self.clock)

    def receive_message(self, received_timestamp):
        """Simulates receiving a message and updating the logical clock."""
        # Update clock to max(current clock, received timestamp) + 1
        self.clock = max(self.clock, received_timestamp) + 1
        print(f"Process {self.pid}: Received message | Updated Clock: {self.clock}")

if __name__ == "__main__":
    # Create three processes with unique IDs
    p1 = LamportClock(1)
    p2 = LamportClock(2)
    p3 = LamportClock(3)

    # Simulate events and message passing
    p1.send_message(p2)  # P1 sends to P2
    p2.event()           # P2 performs an internal event
    p1.send_message(p3)  # P1 sends to P3
    p2.event()           # P2 performs another internal event
    p1.event()           # P1 performs an internal event
    p3.event()           # P3 performs an internal event
    p3.send_message(p1)  # P3 sends to P1
```